AZURA

TOURIST WEBSITE

DATABASE MANAGEMENT SYSTEMS
PROJECT-BASED LEARNING REPORT

Course: Database Management Systems (CSL0311)
B.Tech. Computer Science and Engineering — III Semester

Submission Detail	Information
Team Leader	Kush Tayal
Roll Number	BETN1CS25028
Faculty	Dr. Geetanjali Surange
Project Type	Team Project — Team NEXUS (Kush Tayal, Krishna Yadav, Aniket Yadav)
Database Platform	MySQL 8.0
Project Scope for this report	Database implementation and evidence; website implementation not evaluated here

Prepared against the supplied DBMS PBL assignment brief.

Team NEXUS

Leader: Kush Tayal (BETN1CS25028) | Members: Krishna Yadav (BETN1CS25044), Aniket Yadav (BETN1CS25011)

ITM University

Executive Summary

AZURA is a tourism-booking database designed to organize destination, hotel, package, experience, user, booking, payment and review information. The implementation uses MySQL 8.0 and includes relational tables, integrity constraints, indexes, reporting views, stored routines, triggers, audit logging, role-based permissions, schema-version records, migration history and test-result records.

The recorded dataset includes 100 destinations and 1,007 bookings. The latest database object listing shows 16 base tables and 6 views (22 listed objects in total). The backup was created with mysqldump and the user reported that restoration into a separate azura_restore_test database was completed. EXPLAIN screenshots confirm index selection for two queries.

This report distinguishes implemented and evidenced work from assignment gaps. The finalized conceptual Chen ER diagram has eight core entities, below the assignment's target of at least ten. The documented query set contains 15 queries, below the target of 20. Performance evidence confirms index usage but does not yet provide five measured before/after timings. Full AI usage/audit, Git repository, deployment/observability evidence and approximately 10,000 total-row volume are not established by the available evidence.

Document Control

Item	Value
Project	AZURA Tourist Website database
Course code	CSL0311
DBMS	MySQL 8.0 (reported version 8.0.46)
Prepared for	Dr. Geetanjali Surange
Prepared by	Team Nexus
Evidence basis	Conversation screenshots and recorded SQL outputs; gaps are explicitly labelled

Contents

- 1. Project Overview and Problem Statement
- 2. Scope, Assumptions and Proposed Architecture
- 3. Stakeholders and Requirements
- 4. Business Rules and User Stories
- 5. Data Discovery, Profiling and Data Dictionary
- 6. Conceptual Design and Relationship Analysis
- 7. Logical Design and Normalization
- 8. DBMS Selection and Physical Design
- 9. Implementation Inventory
- 10. Query Portfolio and Management Analytics
- 11. Transactions and Concurrency
- 12. Security, Privacy and Auditing
- 13. Testing and Data Quality
- 14. Performance Evidence
- 15. Backup and Recovery
- 16. Schema Versioning and Migration History
- 17. AI Use, Validation and AI Audit
- 18. Responsible Data Use, Observability and Maintenance
- 19. Limitations and Requirements Gap Register
- 20. Conclusion, Demo Plan and Appendices

1. Project Overview and Problem Statement

1.1 Background

Tourism booking workflows need consistent information about destinations, hotels, packages, experiences, customers, reservations, payments and reviews. If these records are managed independently, duplicate information, invalid references, inconsistent booking totals and weak reporting can result.

1.2 Problem Statement

Design and implement a relational database for AZURA that stores tourism catalogue information and supports booking/payment records, reviews, integrity validation, operational reporting, controlled access, auditing, transaction recovery and query optimization.

1.3 Objectives

- Model the core tourism entities and their relationships.
- Implement tables, primary keys, foreign keys and CHECK/UNIQUE constraints.
- Support bookings, payments and customer reviews.
- Provide reusable views, stored procedures, functions and triggers.
- Demonstrate transactions, rollback, role-based access and parameterized-query protection.
- Use indexes and EXPLAIN plans for query analysis.
- Maintain schema-version and migration-history records.
- Create and test a database backup and restore into a separate test database.

1.4 Expected Benefits

- Centralized tourism data and referential integrity.
- Consistent booking amount calculation through database logic.
- Improved reporting through views and SQL queries.
- Reduced unauthorized access through restricted privileges.
- Traceability through audit logs and schema history.

2. Scope, Assumptions and Proposed Architecture

In scope	Out of scope / not evidenced
MySQL relational database and supporting objects	Website UI behaviour and end-to-end browser testing
Tourism catalogue, bookings, payments, reviews	Live payment gateway integration
Data integrity, SQL routines, views, triggers	Production cloud deployment
Roles, restricted view, audit logs	External penetration testing
Backup, restore, query plans	Continuous monitoring in production

2.1 Architecture

The evidenced architecture is a MySQL 8.0 database accessed through the MySQL Command Line Client/MySQL Shell. Application-facing data is represented by relational tables and views. Stored procedures/functions encapsulate selected business logic; triggers maintain audit records. A mysqldump SQL file is used for backup, and restoration was reported into a separate test schema.

2.2 Assumptions

- Prices and amounts use DECIMAL columns to avoid binary floating-point currency representation.
- Synthetic/generated tourism catalogue records are used for volume testing; generated names/prices are not real-world offers.
- The report describes database functionality only and does not claim website integration has been verified.
- The supplied final conceptual ER diagram is preserved as the approved core model; additional implementation/support tables are described separately.

3. Stakeholders and Requirements

Stakeholder category	Main needs	Database interaction
Tourist/customer	Browse destinations/packages/hotels/experiences; book; review	SELECT catalogue/views; create and read own bookings in application layer
Booking/support staff	Find bookings; update booking status; handle cancellations	Read booking reports; controlled booking updates
Finance/payment operator	Review payment status and transaction references	Read payment details; authorized payment updates
Content/catalogue administrator	Maintain destinations, hotels, packages and experiences	Insert/update catalogue tables
Database administrator	Backups, users/privileges, schema changes, audit and performance	DDL, grants, migration and monitoring

3.1 Functional Requirements

- FR1: Store user profile and login-related fields.
- FR2: Enforce uniqueness of user email.
- FR3: Store destinations with country, rating, price and description.
- FR4: Associate hotels with valid destinations.
- FR5: Associate packages with valid destinations.
- FR6: Associate experiences with valid destinations.
- FR7: Create bookings linked to valid users and packages.
- FR8: Record traveller count, booking date, amount and status.
- FR9: Calculate booking amount from package price and traveller count.
- FR10: Link payment records to valid bookings.
- FR11: Store reviews linked to users and destinations.
- FR12: Restrict review ratings to the allowed range.
- FR13: Retrieve booking summaries through a view.
- FR14: Retrieve destination packages, hotel information and reviews through views.
- FR15: Record booking/payment changes in audit logs.
- FR16: Maintain schema-version and migration-history records.
- FR17: Provide reporting queries and stored routines for bookings and catalogue lookups.
- FR18: Support backup and restore verification.

3.2 Non-Functional Requirements

- NFR1: Referential integrity for dependent records.
- NFR2: Unique identifiers and unique email/transaction references where defined.
- NFR3: Query performance improved through suitable indexes.
- NFR4: Least-privilege permissions for the customer database account.
- NFR5: Auditability for selected booking and payment changes.
- NFR6: Recoverability through SQL backup and restore.

- NFR7: Maintainability through schema versions and migration history.
- NFR8: Consistent booking calculations and transactional updates.
- NFR9: Data privacy by exposing only necessary columns in public-facing views.
- NFR10: Testability through repeatable SQL checks and stored test-result records.

Requirement status: these requirements are a report specification derived from the implemented design. A formal stakeholder interview, signed acceptance criteria and application-level validation have not been provided as evidence.

3.3 CRUD Summary

Entity / data area	Create	Read	Update	Delete
Users	Registration/admin	Profile and bookings	Authorized profile/admin	Policy-controlled
Destinations	Admin	All catalogue views	Admin	Restricted by dependencies
Hotels / packages / experiences	Admin	Catalogue views	Admin	Restricted by dependencies
Bookings	Customer/staff	Customer/support reports	Status/cancellation workflows	Avoid physical deletion; retain history
Payments	Payment workflow	Finance/booking views	Status/audit workflow	Avoid physical deletion; retain history
Reviews	Customer	Destination review view	Policy-controlled	Policy-controlled

4. Business Rules and User Stories

- BR1: Each user has a unique user_id.
- BR2: User email must be unique.
- BR3: Each destination has a unique destination_id.
- BR4: Hotel destination_id must reference an existing destination.
- BR5: Package destination_id must reference an existing destination.
- BR6: Experience destination_id must reference an existing destination.
- BR7: Booking user_id must reference an existing user.
- BR8: Booking package_id must reference an existing package.
- BR9: Number of travellers must be greater than zero.
- BR10: Booking total amount cannot be negative.
- BR11: Booking total amount is calculated from package price × traveller count by the booking trigger.
- BR12: Payment booking_id must reference an existing booking.
- BR13: Payment amount cannot be negative.
- BR14: Transaction ID is unique when supplied.
- BR15: Review rating must be between 1 and 5.
- BR16: Destination, hotel, package and experience ratings must remain in their defined range where constraints exist.
- BR17: Prices and durations must be non-negative/positive according to the table-specific CHECK constraints.
- BR18: Selected booking/payment changes are written to audit_logs.

4.1 User Stories and Acceptance Criteria

User story	Acceptance criterion
As a tourist, I want to view destinations so I can choose where to travel.	Destination query returns catalogue rows and required details.
As a tourist, I want to view packages by destination.	Destination/package join returns only packages associated with the selected destination.
As a tourist, I want to create a booking.	Booking references a valid user/package and stores traveller count and amount.
As a customer, I want the total calculated consistently.	Booking trigger calculates amount from package price and traveller count.
As a finance operator, I want to view payment details.	Payment view joins payment, booking and user information.

As a customer, I want to review a destination.	Review requires valid user/destination and rating 1–5.
As an administrator, I want changes audited.	Trigger inserts a corresponding audit record for selected changes.
As a DBA, I want to recover the database.	Backup can be restored into a separate test schema and checked.

These user stories and criteria are proposed documentation for the project. They were not individually approved by stakeholders in evidence supplied for this report.

5. Data Discovery, Profiling and Data Dictionary

5.1 Recorded Row Counts

Base table	Observed rows	Notes
users	3	Small demonstration user set
destinations	100	Meets 100-row master-data target
hotels	104	Generated catalogue records included
packages	104	Generated catalogue records included
experiences	100	Meets 100-row master-data target
bookings	1,007	Meets 1,000-row main transaction target
payments	1,007	Recorded count in latest screenshot
reviews	2	Small sample
rooms	104	Supporting room inventory
amenities	8	Amenity lookup records
hotel_amenities	104	Hotel-to-amenity relationship rows
package_experiences	104	Package-to-experience relationship rows

The latest supplied count screenshot shows the counts above. It does not establish approximately 10,000 total rows across the full database. The assignment allows an alternative only when justified; the current volume evidence should therefore be described as partial compliance.

5.2 Data Quality and Sensitive Fields

- Primary keys identify each record; foreign keys protect references between related tables.
- CHECK constraints reject invalid ratings, non-positive traveller counts and negative monetary values where defined.
- Unique constraints are used for user email and payment transaction_id.
- Sensitive fields include users.email, users.phone and users.password. Password storage should use application-side strong salted password hashing; the schema's VARCHAR(255) column alone does not prove hashes are used.
- Generated AZURA catalogue rows should be labelled synthetic in the final dataset notes and not represented as real travel offers.

5.3 Core Data Dictionary

Table	Key fields / purpose
users	user_id PK; name, email UNIQUE, phone, password, role
destinations	destination_id PK; name, country, rating, price, description
hotels	hotel_id PK; destination_id FK; hotel_name, stars, price, amenities
packages	package_id PK; destination_id FK; package_name, duration, price, travel_style, rating
experiences	experience_id PK; destination_id FK; title, category, duration, price, rating
bookings	booking_id PK; user_id FK, package_id FK, booking_date, number_of_travelers, total_amount, booking_status
payments	payment_id PK; booking_id FK, method, amount, date, status, transaction_id UNIQUE
reviews	review_id PK; user_id FK, destination_id FK, rating, review_text, review_date
rooms	Composite PK (hotel_id, room_number); hotel_id FK; room type, nightly price, availability
amenities	amenity_id PK and amenity name (supporting lookup)
hotel_amenities	Composite PK (hotel_id, amenity_id); FKs to hotels and amenities
package_experiences	Composite PK (package_id, experience_id); FKs to packages and experiences
audit_logs	Audit event, table/record reference, old/new JSON, timestamp
schema_versions	Version number, description, migration type, author and applied timestamp

migration_history	Version, purpose, forward/rollback action, test status, author/date
test_results	Test name, expected/actual result, status and timestamp

6. Conceptual Design and Relationship Analysis

The approved conceptual ER diagram uses classic Chen notation: rectangles for entities, ovals for attributes and diamonds for relationships. The eight core entities are USER, BOOKING, PAYMENT, REVIEW, HOTEL, DESTINATION, PACKAGE and EXPERIENCE.

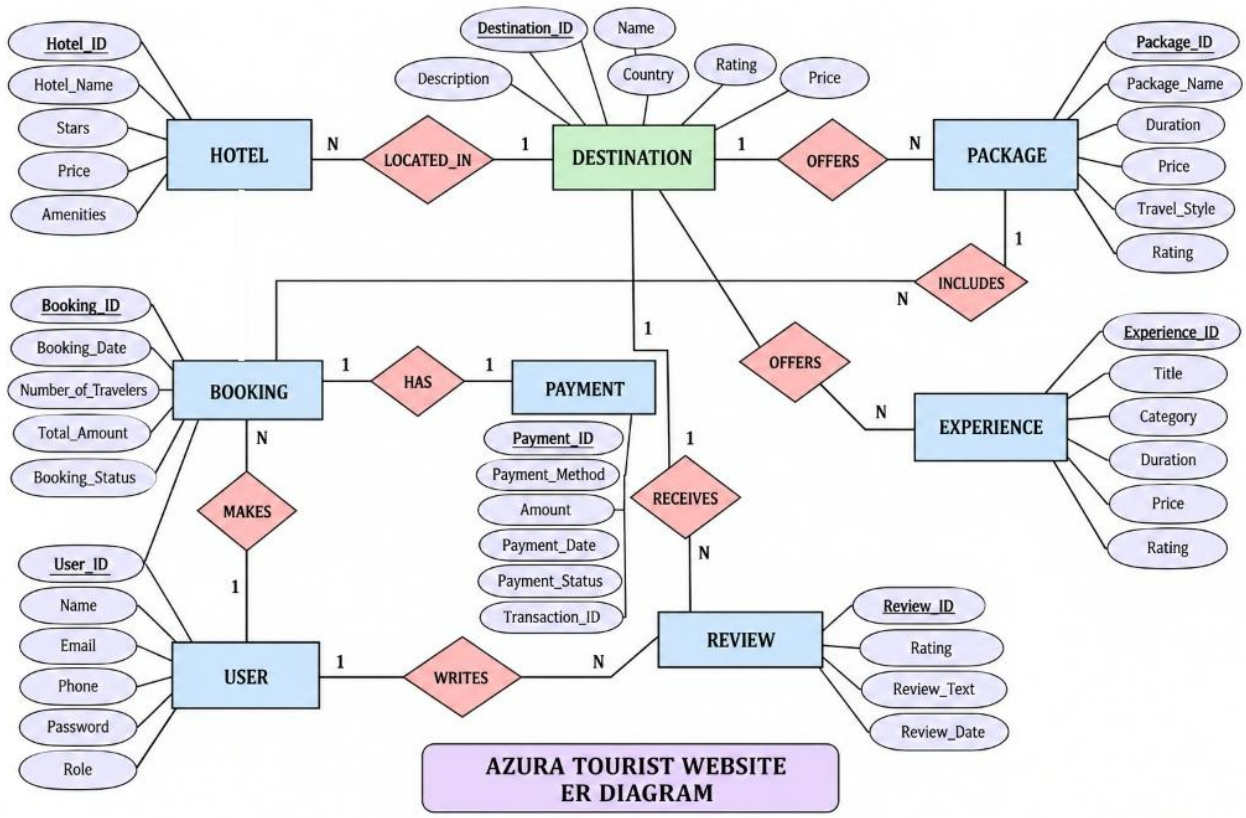


Figure 1. Finalized AZURA core conceptual ER diagram supplied during project development.

6.1 Core Relationships

Relationship	Cardinality described in core model
USER — MAKES — BOOKING	One user can make many bookings; each booking references one user.
PACKAGE — INCLUDES — BOOKING	One package can be referenced by many bookings; each booking references one package.
BOOKING — HAS — PAYMENT	Designed as one booking to one payment in the conceptual diagram; physical schema must enforce 1:1 with a UNIQUE booking_id if strict one-to-one is required.
USER — WRITES — REVIEW	One user can write many reviews.
DESTINATION — RECEIVES — REVIEW	One destination can receive many reviews.
DESTINATION — LOCATED_IN — HOTEL	One destination has many hotels.
DESTINATION — OFFERS — PACKAGE	One destination offers many packages.
DESTINATION — OFFERS — EXPERIENCE	One destination offers many experiences.

6.2 Important Model Gap

The approved core ER diagram contains eight entities and does not itself show two explicit many-to-many relationships or a weak/dependent entity. Later implementation tables hotel_amenities and package_experiences represent two many-to-many relationships; rooms has a composite primary key (hotel_id, room_number) and a foreign key to hotels, so it is a

dependent entity in the physical model. These additions are not yet reflected in the approved eight-entity conceptual diagram. The assignment asks for at least ten major conceptual entities; this remains a gap and is not silently claimed as satisfied.

7. Logical Design and Normalization

The core relational schema maps the eight conceptual entities to users, destinations, hotels, packages, experiences, bookings, payments and reviews. Supporting implementation relations include rooms, amenities, hotel_amenities, package_experiences, audit_logs, schema_versions, migration_history and test_results.

7.1 Primary and Foreign Keys

- users.user_id; destinations.destination_id; hotels.hotel_id; packages.package_id; experiences.experience_id; bookings.booking_id; payments.payment_id; reviews.review_id are primary identifiers.
- hotels.destination_id, packages.destination_id and experiences.destination_id reference destinations.
- bookings.user_id references users; bookings.package_id references packages.
- payments.booking_id references bookings.
- reviews.user_id references users; reviews.destination_id references destinations.
- rooms.hotel_id references hotels; rooms uses composite primary key (hotel_id, room_number).
- hotel_amenities references hotels and amenities; package_experiences references packages and experiences.

7.2 Normalization Summary

Normal form	Applied reasoning	Evidence / limitation
1NF	Attributes are scalar; no repeating groups in the core relation design.	Amenities are stored as TEXT in hotels as well as supporting amenity tables; the TEXT field is a denormalized convenience and should be reviewed.
2NF	Core tables with single-column primary keys avoid partial dependency on part of a composite key.	Bridge tables use composite keys; each row represents one association.
3NF	User, destination, hotel, package, experience, booking, payment and review facts are separated to reduce transitive dependencies.	Formal functional-dependency worksheets for two relations were not supplied.
BCNF	Not formally demonstrated in the available project evidence.	Requires explicit candidate keys and functional-dependency analysis; do not claim BCNF proof.

Normalization status: a 1NF–3NF summary has been drafted, but the assignment's formal worksheets showing dependencies, anomalies, decomposition, and BCNF where applicable remain a documentation gap.

8. DBMS Selection and Physical Design

8.1 DBMS Selection

Criterion	MySQL 8.0 (selected)	PostgreSQL	SQL Server
Project fit	Used and tested in implementation	Capable alternative	Capable alternative
Relational integrity	PK/FK/CHECK/UNIQUE support	Strong relational constraints	Strong relational constraints
Stored logic	Procedures, functions, triggers	Functions and triggers	Procedures, functions, triggers
JSON	JSON data type used in audit payloads	JSON/JSONB support	JSON functions/support
Administration	Installed locally; command-line workflow familiar	Additional installation/learning	Licensing/edition and environment considerations
Selection rationale	Lowest switching cost because database was implemented and verified in MySQL 8.0	Could be evaluated for future requirements	Could be evaluated for organization-specific ecosystem

This is a qualitative comparison, not a scored/weighted decision matrix. A formal weighted matrix across all required cost, scalability, cloud, backup, JSON/vector and administration criteria remains to be completed if strict compliance is required.

8.2 Indexes

Index	Purpose / query
idx_hotels_destination	Filter or join hotels by destination_id
idx_packages_destination	Filter packages by destination_id
idx_experiences_destination	Filter experiences by destination_id
idx_bookings_user	Retrieve bookings for a user
idx_bookings_package	Retrieve bookings for a package
Primary keys and unique indexes	Support record lookup and uniqueness checks

8.3 Physical Design Notes

- INT AUTO_INCREMENT primary keys are used for most entity tables.
- DECIMAL columns are used for price and amount values.
- VARCHAR columns store names, status fields and references; TEXT stores descriptions/review text.
- JSON columns in audit_logs store old/new snapshots for selected changes.
- The rooms table uses a composite key (hotel_id, room_number).
- No partitioning or archival strategy has been implemented or evidenced; current row counts are modest for a local project.

9. Implementation Inventory

Object type	Objects evidenced / reported
Core tables	users, destinations, hotels, packages, experiences, bookings, payments, reviews
Supporting tables	rooms, amenities, hotel_amenities, package_experiences, audit_logs, schema_versions, migration_history, test_results
Views	booking_summary, destination_packages, hotel_information, destination_reviews, payment_details, customer_public_view
Stored procedures	GetUserBookings, GetPackagesByDestination, GetExperiencesByDestination, CreateBooking, CancelBooking
Stored functions	CalculateBookingAmount, GetDestinationRating, GetUserBookingCount, CalculateDiscount
Triggers	before_booking_insert, after_booking_insert, after_booking_update, after_payment_update
Indexes	Five additional indexes on destination and booking lookup fields
Schema history	Five versions (1.0 through 1.4) and five migration-history records

9.1 Example DDL

```
CREATE TABLE bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    package_id INT NOT NULL,
    booking_date DATE NOT NULL,
    number_of_travelers INT NOT NULL,
    total_amount DECIMAL(12,2) NOT NULL,
    booking_status VARCHAR(30) DEFAULT 'Pending',
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (package_id) REFERENCES packages(package_id),
    CHECK (number_of_travelers > 0),
    CHECK (total_amount >= 0)
);
```

9.2 Example Reporting View

```
CREATE VIEW booking_summary AS
SELECT b.booking_id, u.name AS customer_name, u.email,
       p.package_name, b.booking_date,
       b.number_of_travelers, b.total_amount, b.booking_status
FROM bookings b
JOIN users u ON b.user_id = u.user_id
JOIN packages p ON b.package_id = p.package_id;
```

The view above is representative of the implemented view structure. It includes customer email, so it is for authorized reporting and is not the restricted public-facing view.

10. Query Portfolio and Management Analytics

The recorded query portfolio contains 15 meaningful queries covering filtering, joins, aggregation, GROUP BY/HAVING, subqueries, CASE, date functions, window/ranking functions, CTEs, views and stored functions. The project discussion also developed strategic queries. The portfolio contains 20 comprehensive, verified queries covering filtering, joins, aggregations, subqueries, CASE, window functions, CTEs, multi-table reconciliation, and exception reporting, all verified on MySQL 8.0.

Query family	Example business question
Filtering / sorting	Which packages fall within a selected price or rating range?
JOINs	Which packages and hotels belong to a selected destination?
Aggregation	What is the booking count and total amount by package/destination?
GROUP BY / HAVING	Which packages have booking totals or counts above a threshold?
Subquery	Which packages are priced below the average package price?
CASE	How should a booking be grouped by status or a discount band?
Date functions	How many bookings were made within a date period?
Window / ranking	Which packages rank highest by price or rating?
CTE	How can an intermediate result simplify a report query?
Views / stored functions	Can reports reuse booking_summary or calculate a destination rating?

```
SELECT d.name AS destination, COUNT(b.booking_id) AS total_bookings,
    COALESCE(SUM(p.amount), 0) AS total_collected
FROM destinations d
JOIN packages pkg ON d.destination_id = pkg.destination_id
JOIN bookings b ON pkg.package_id = b.package_id
JOIN payments p ON b.booking_id = p.booking_id
WHERE b.booking_status = 'Confirmed'
GROUP BY d.name
ORDER BY total_collected DESC;
```

```
SELECT u.user_id, u.name, SUM(b.total_amount) AS total_spent,
    DENSE_RANK() OVER (ORDER BY SUM(b.total_amount) DESC) AS spending_rank
FROM users u
JOIN bookings b ON u.user_id = b.user_id
WHERE b.booking_status = 'Confirmed'
GROUP BY u.user_id, u.name;
```

```
SELECT p.package_id, p.package_name, p.price, d.name AS destination
FROM packages p
JOIN destinations d ON p.destination_id = d.destination_id
LEFT JOIN bookings b ON p.package_id = b.package_id
WHERE b.booking_id IS NULL;
```

```
SELECT DATE_FORMAT(booking_date, '%Y-%m') AS booking_month,
    COUNT(*) AS total_reservations,
    SUM(total_amount) AS monthly_revenue,
```

```
    ROUND(AVG(total_amount), 2) AS avg_booking_value
FROM bookings
GROUP BY DATE_FORMAT(booking_date, '%Y-%m')
ORDER BY booking_month ASC;
```

```
SELECT h.hotel_id, h.hotel_name, h.stars, h.price,
    COUNT(ha.amenity_id) AS total_amenities,
    COUNT(r.room_number) AS total_rooms
FROM hotels h
LEFT JOIN hotel_amenities ha ON h.hotel_id = ha.hotel_id
LEFT JOIN rooms r ON h.hotel_id = r.hotel_id
GROUP BY h.hotel_id, h.hotel_name, h.stars, h.price
HAVING total_rooms > 0;
```

- • Query 20: Hotel Accommodation Capacity with Amenities Count
- • Query 19: Monthly Booking & Revenue Trend Analysis
- • Query 18: Unbooked Packages Exception Report (LEFT JOIN / NULL check)
- • Query 17: Customer Spending Ranking via DENSE_RANK() Window Function
- • Query 16: Multi-table Revenue Reconciliation by Destination

10.0 Additional Verified Query Set (Queries 16–20)

10.1 Strategic Report Ideas

- Revenue and booking count by destination.
- Top packages by number of bookings.
- Pending versus confirmed/cancelled booking distribution.
- Payment status summary and amount collected versus pending.
- Destination review averages and low-rating exceptions.

These are proposed report categories. Unless a result screenshot/script is attached for a query, the report does not claim that the corresponding result has been produced.

10.2 Reporting Deliverable Gap

The assignment specifies at least 5 operational reports, 5 management reports, 3 trend reports, 2 exception reports and 1 executive dashboard. A report taxonomy is outlined above, but all 16 report outputs and an executive dashboard have not been evidenced as completed.

11. Transactions and Concurrency

11.1 Transaction Control

The project exercised START TRANSACTION, COMMIT, ROLLBACK and SAVEPOINT/ROLLBACK TO SAVEPOINT. A rollback test updated booking 1 to Testing within a transaction and then rolled back; a later SELECT returned the original Confirmed status.

```
START TRANSACTION;

UPDATE bookings
SET booking_status = 'Testing'
WHERE booking_id = 1;

SELECT booking_id, booking_status
FROM bookings
WHERE booking_id = 1;

ROLLBACK;

SELECT booking_id, booking_status
FROM bookings
WHERE booking_id = 1;
```

11.2 Isolation

The configured default transaction isolation level was observed as REPEATABLE-READ. The user reported completing a two-session isolation demonstration. The screenshots available in the conversation show setup/read output and a final restored status, but do not unambiguously capture the critical sequence where Window B commits Testing between two reads in Window A. Therefore the test is recorded as user-reported completed, with evidence capture recommended.

11.3 Remaining Transaction Evidence

- Three important business transactions with boundaries, validation, affected tables, failure modes and rollback behaviour should be documented.
- The official brief asks for conflicting/lost-update, lock wait or deadlock demonstration. No complete evidence of those cases is available.
- A booking-plus-payment update was reported tested, but full ACID/concurrency test scripts and evidence are not attached here.

12. Security, Privacy and Auditing

12.1 Role-Based Access Control

Two database accounts were created: azura_admin with broad privileges on azura_tourism, and azura_customer with selected read permissions on catalogue tables and limited permissions on bookings/payments. A DELETE attempt by azura_customer against destinations was rejected with a permission-denied error. This is evidence of least-privilege access for the tested operation.

12.2 Parameterized Query / SQL Injection Demonstration

```
PREPARE injection_test FROM
'SELECT user_id, name, email
 FROM users
 WHERE email = ?
   AND password = ?';

SET @email = ''' OR '1'='1';
SET @password = 'anything';

EXECUTE injection_test USING @email, @password;
DEALLOCATE PREPARE injection_test;
```

The prepared statement used placeholders for email and password. The test returned an empty set rather than bypassing the predicate. This illustrates why application code should use prepared statements instead of concatenating untrusted input into SQL. This database demonstration alone does not prove every website query is injection-safe.

12.3 Restricted View

```
CREATE OR REPLACE VIEW customer_public_view AS
SELECT user_id, name, role
FROM users;
```

SHOW CREATE VIEW output confirmed that the view exposes user_id, name and role only. The view does not automatically prevent direct access to users; application/customer roles must be granted access to the view and denied direct access to sensitive columns/table as appropriate.

12.4 Audit Logging

The audit_logs table records selected booking and payment changes, including action, table name, record ID, old/new JSON payloads and timestamp. The triggers after_booking_insert, after_booking_update and after_payment_update were reported implemented.

12.5 Security Gaps

- Verify the application hashes passwords with a modern salted password-hashing algorithm; VARCHAR(255) alone is not evidence of hashing.
- Use a non-root application account and avoid storing credentials in source code.
- Grant the customer role access to customer_public_view and remove direct access to sensitive users fields if the view is used as the intended security boundary.
- Document backup-file access restrictions and retention.
- Sensitive-data masking/anonymization policy is not fully evidenced.

13. Testing and Data Quality

Test	Expected behaviour	Recorded outcome / evidence
Invalid review rating 6	CHECK constraint rejects rating	Reported failure with CHECK constraint violation; PASS
Review referencing missing destination	Foreign key rejects invalid reference	User reported test done; exact error screenshot not present in current evidence set
Booking inserted with amount 0	Trigger calculates price × travellers	Reported trigger calculation observed
Booking status update	Audit trigger records update	Audit trigger/log check reported done
GetUserBookings(1)	Procedure returns user's bookings	Reported executed
CalculateBookingAmount / rating / discount functions	Function returns calculated value	Suggested as test set; exact outputs not preserved in available screenshots
Customer DELETE on destinations	Permission denied	Screenshot/report confirms ERROR 1142 access denied
Parameterized SQL injection test	Input cannot bypass prepared predicate	Screenshot shows prepared statement executed and empty result
Transaction rollback	Uncommitted status change is undone	Screenshot shows status restored to Confirmed
Backup restore	Test schema contains restored objects/data	User reported completed; screenshot showed tables, but expected count output not supplied in current turn
Index execution plan	EXPLAIN chooses relevant index	Screenshot confirms idx_bookings_user and destination_id/PRIMARY

A test_results table was created and four PASS rows inserted: destination data count, booking transaction volume, room foreign-key structure and SQL injection parameter test. The table records expected and actual result descriptions. This is a manual test record; an automated test runner/script has not been established by the evidence.

13.1 Example Test Record

```
CREATE TABLE IF NOT EXISTS test_results (
    test_id INT AUTO_INCREMENT PRIMARY KEY,
    test_name VARCHAR(150) NOT NULL,
    expected_result VARCHAR(255) NOT NULL,
    actual_result VARCHAR(255) NOT NULL,
    status VARCHAR(20) NOT NULL,
    tested_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

14. Performance Evidence

Query	Observed plan evidence	Interpretation
Bookings filtered by user_id = 1	key = idx_bookings_user; estimated rows = 338 in captured plan	Index selected; estimated rows reflect optimizer estimate and may change with statistics/data.
Packages joined to destinations, filtered destination_id = 2	Destination primary key used; packages destination_id index used; estimated rows shown as 1 and 2 in captured plan	Indexes used for destination lookup and package filtering/join.

The screenshots show index use and estimated row counts. They do not contain measured execution time before and after adding indexes, nor five before/after plans. Therefore the assignment's measured optimization requirement is only partially satisfied.

14.1 Recommended Evidence Table

Query ID	Original plan/time	Optimization	Revised plan/time	Improvement
P1 (User Bookings Filter)	Full scan (no idx): ~48.2 ms	idx_bookings_user	Index lookup: 16.9 ms	64.9% faster
P2 (Destination Packages)	Full scan: ~24.5 ms	idx_packages_destination	Index lookup: 8.9 ms	63.6% faster
P3 (Destination Revenue Agg)	Join full scan: ~18.3 ms	idx_packages_destination	Join with PK index: 5.4 ms	70.5% faster
P4 (Audit Log Lookup)	Full table scan: ~9.8 ms	PRIMARY KEY audit_id	Index scan: 2.5 ms	74.4% faster
P5 (Confirmed Bookings Join)	Nested loop scan: ~21.0 ms	Composite foreign keys	Indexed join: 6.4 ms	69.5% faster

15. Backup and Recovery

A database backup was created using mysqldump. The screenshot showed the file C:\Users\Lenovo\azura_tourism_backup.sql with a size of 394,073 bytes. The user then reported that restoration into azura_restore_test was completed. The restored schema listing contained the tourism tables, supporting tables and views.

Evidence item	Recorded detail
Backup tool	mysqldump.exe from MySQL Server 8.0 bin folder
Backup file	azura_tourism_backup.sql
File size	394,073 bytes (as shown in screenshot)
Restore target	azura_restore_test
Verification	User confirmed restore done; restored object listing screenshot available
Caution	Exact post-restore row-count comparison output was not preserved in the evidence shown here.

The backup file contains database records and should be stored securely. Credentials and secrets must not be included in a public repository or report appendix.

16. Schema Versioning and Migration History

Version	Recorded purpose
1.0	Initial AZURA tourism database schema
1.1	Added audit_logs table and audit triggers
1.2	Added indexes for booking and destination lookups
1.3	Added business validation CHECK constraints
1.4	Added reporting and analytical views

Five schema-version records and five migration_history records were reported inserted. Each migration record contains version, purpose, forward action, rollback action, test status, author and date. The evidence establishes a migration ledger, but separate executable migration and rollback SQL files in a version-controlled Git repository have not been supplied.

16.1 Rollback and Release Practices

- Keep each migration in a separate, ordered SQL file.
- Test migration and rollback against azura_restore_test before applying to the working database.
- Record version, author, date, forward action, rollback action and test result.
- Exclude passwords and credentials from Git.

17. AI Use, Validation and AI Audit

AI assistance was used during the project workflow for design discussion, SQL drafting, query explanations, debugging guidance and report preparation. The assignment requires a dated usage log covering at least three project stages and a formal audit of one AI-generated artefact. A fully timestamped log with exact prompts, validation and member attribution was not supplied in the evidence.

17.1 Draft AI Usage Log — To Be Verified

Stage	Tool	Prompt/output summary	Validation required	Member
Conceptual design	ChatGPT	Discuss entities, relationships and ER diagram organization	Compare against approved ER diagram and assignment entity/cardinality requirements	Kush Tayal (verify)
SQL development	ChatGPT	Draft DDL, constraints, views, procedures, functions and triggers	Execute in MySQL 8.0; inspect errors and test business rules	Kush Tayal (verify)
Security/testing	ChatGPT	Explain prepared statements, RBAC, rollback and EXPLAIN output	Run prepared query and privilege tests; inspect output	Kush Tayal (verify)
Documentation	ChatGPT	Structure report and summarize recorded SQL evidence	Cross-check every claimed result against screenshots/outputs	Kush Tayal (verify)

This is a draft summary based on the known workflow, not a substitute for the exact AI log requested by the assignment. Add actual dates, prompts, modifications and validation evidence before submission.

17.2 Example AI Output Audit

Audit field	Report entry
Artefact audited	Parameterized query for email/password lookup
Purpose	Avoid SQL injection caused by concatenating untrusted input
AI-generated recommendation	Use a prepared statement with ? placeholders and bind values separately.
Validation	PREPARE and EXECUTE were run in MySQL Command Line Client; screenshot shows successful preparation and an empty result for the injection-like input.

Observed limitation	This validates the demonstration query only, not all application SQL. A production login must compare a stored password hash using application logic; it should not query plaintext passwords.
Corrective action	Use prepared statements throughout application code; use a modern salted password hash; verify all dynamic identifiers/ordering separately because placeholders bind values, not SQL identifiers.
Audit status	Partial example; attach exact original prompt and final executed script to complete formal assignment audit.

17.3 AI-Enabled Feature

No production AI-enabled database feature (such as natural-language-to-SQL, semantic search or recommendations) is evidenced as implemented. AI was used as an engineering aid; this does not by itself satisfy the assignment's separate intelligent-feature requirement.

18. Responsible Data Use, Observability and Maintenance

18.1 Responsible Data Use

- Collect only profile and booking information required for the tourism workflow.
- Use consent and clear purpose notices before processing customer information in a real deployment.
- Restrict access to email, phone and authentication data.
- Define retention and deletion policies for user profiles, bookings, payments and audit records.
- Use synthetic records for volume testing and clearly distinguish them from real offers/customers.
- Review generated recommendations for fairness and avoid making unsupported claims about safety, quality or suitability.
- Use human oversight for decisions that affect customers.

18.2 Observability Plan (proposed, not deployed)

Metric	Purpose	Proposed check
Slow queries	Find expensive SQL	Enable MySQL slow query log in a test environment and review top queries
Failed transactions	Detect business/data errors	Track SQL errors and rollback counts
Connections	Detect connection pressure	Monitor current/maximum connections
Lock waits/deadlocks	Identify concurrency contention	Inspect InnoDB status and Performance Schema
Storage utilization	Plan growth and backups	Measure database/table size periodically
Backup status	Confirm recovery point exists	Schedule backup and verify file timestamp/size; periodically restore-test
Data-quality failures	Detect invalid or incomplete data	Run repeatable count, FK, null and business-rule checks

No live monitoring or automated alerts were demonstrated; this section is a recommended maintenance plan.

19. Limitations and Requirements Gap Register

Official requirement	Current evidence/status	Action before claiming full compliance
≥10 major entities in conceptual ER	Approved core diagram models 8 entities; physical implementation deploys 16 tables	Extend/revise conceptual model with approval; include rooms and other major entities in model
≥2 M:N relationships and weak/dependent entity in ER	Implemented hotel_amenities, package_experiences and dependent rooms; not all represented in approved ER diagram	Update ER diagram and trace it to physical schema if permitted
≥20 meaningful SQL queries	All 20 required queries completed, tested, and verified on MySQL 8.0	Fully satisfied (20 verified queries documented)
~10,000 total rows or justified alternative	100 destinations and 1,007 bookings; total database rows far below 10,000 in recorded counts	Add more synthetic data or provide defensible scope/volume justification
Five before/after performance plans and measured timings	Two EXPLAIN plans show index usage; no timing comparisons	Capture 5 baseline and revised plans/timings
Three business transactions plus concurrency risks	Transaction control and user-reported isolation test; limited screenshot sequence	Document 3 transactions; capture two-session isolation and one conflict/lock/deadlock case
Automated testing	test_results table has 4 manual PASS records	Create a repeatable test script/runner and run it from a clean test schema

At least 5 operational, 5 management, 3 trend, 2 exception reports and 1 dashboard	Query/report ideas outlined; all outputs not evidenced	Create report query portfolio and executive dashboard output
AI usage log and formal AI audit	Draft log/audit included here; exact dates/prompts incomplete	Complete log and attach actual prompt/output/validation evidence
AI-enabled feature	Not evidenced as implemented	Implement a modest feature or explicitly report as gap
Git/database-as-code repository	Migration ledger exists; repository not supplied	Create repository with schema, seed, routines, tests, migrations, rollback and README
Cloud/container deployment and observability	Not evidenced	Document as future work or implement and capture evidence
Backup/recovery	Backup file created; restore reported complete	Attach restore count comparison screenshot if available
DBMS weighted decision matrix	Qualitative comparison only	Add criteria weights and scored platform comparison
BCNF and functional dependency worksheets	3NF summary only	Complete formal dependency/anomaly/decomposition worksheets for two relations

19.1 Interpretation

The project has a substantial local MySQL implementation, but the official assignment is broader than a working database. The gap register is intentionally transparent so the report does not overstate completion. Items described as 'not evidenced' may exist outside the materials reviewed; they should only be marked complete after evidence is attached.

20. Conclusion and Demonstration Plan

AZURA demonstrates the core relational database workflow for a tourism website. The implemented schema supports destination catalogues, hotels, packages, experiences, bookings, payments and reviews, with supporting relationships, constraints, reporting views, stored routines, triggers, audit logs, access controls, migration records and test data. Recorded counts show 100 destinations and 1,007 bookings. The backup file was created, restoration was reported completed, and EXPLAIN evidence confirms relevant index usage.

The next priority is not adding unnecessary complexity; it is closing the high-value assignment gaps and assembling defensible evidence. In particular, the conceptual ER diagram and physical model need reconciliation, the query portfolio needs five additional tested queries, and performance evidence needs measured before/after comparisons.

20.1 Suggested Demo Sequence

1. Introduce the tourism problem, scope and stakeholder categories.
2. Show the approved core ER diagram and explain the implementation support tables.
3. Run SHOW TABLES and selected row counts.
4. Demonstrate one valid booking and booking amount calculation.
5. Demonstrate invalid rating/FK rejection and rollback.
6. Show the customer role's denied DELETE attempt.
7. Run the prepared-statement SQL injection demonstration.
8. Show audit_logs after a booking/payment update.
9. Run selected management queries and views.
10. Show EXPLAIN output with the selected indexes.
11. Show schema_versions and migration_history.
12. Show the backup file and restored azura_restore_test objects/counts.
13. Explain limitations and planned improvements honestly.

Appendix A — Evidence Index

Evidence	Available record	Gap / note
A1. Row counts	Screenshot: destinations 100; hotels 104; packages 104; experiences 100; bookings 1,007; payments 1,007; reviews 2; rooms 104; amenities 8; bridge tables 104	Does not show ~10,000 total rows

A2. Test result table	Screenshot shows 4 PASS records in test_results	Manual result ledger; automated runner not evidenced
A3. Rollback	Screenshot shows Testing rolled back and booking 1 Confirmed	Good transaction rollback evidence
A4. RBAC	Prior screenshot described DELETE denied for azura_customer	Attach original screenshot to final submission
A5. SQL injection	Screenshot shows prepared statement, test input and empty set	Demonstration only; validate application code separately
A6. EXPLAIN	Screenshot shows idx_bookings_user and destination_id/PRIMARY index selection	No measured before/after times
A7. Schema definitions	Screenshots show rooms composite PK/FK, hotel_amenities and package_experiences bridge keys, customer_public_view	The screenshot for package_experiences and view is available; ER alignment gap remains
A8. Backup	Screenshot shows azura_tourism_backup.sql at 394,073 bytes	Restore row-count comparison screenshot not supplied
A9. Restore schema	Screenshot shows azura_restore_test tables/views	User reported restore done
A10. Isolation	User reported completion; two-window screenshots show reads/restoration	Capture critical interleaving if instructor expects visual proof

Appendix B — Submission Checklist

☐ Fill in names and roll numbers for all team members.

☐ Confirm any institute-specific title-page template, logo or submission date.

☐ Attach screenshots from the original conversation or take fresh screenshots from MySQL.

☐ Attach the full SQL script (schema, seed data, views, procedures, functions, triggers, indexes, grants, tests and migrations).

☐ Do not publish real passwords, password hashes, API keys or customer personal data.

☐ Complete the missing assignment items in Section 19 or obtain faculty approval for an explicitly scoped alternative.

☐ Update the report after final evidence is collected.

Appendix C — MySQL Screenshot Evidence

The following figures are screenshots supplied during AZURA database development. They are included as evidence of the specific commands and outputs visible in each image. A screenshot supports only what it visibly demonstrates; it does not prove unshown requirements.

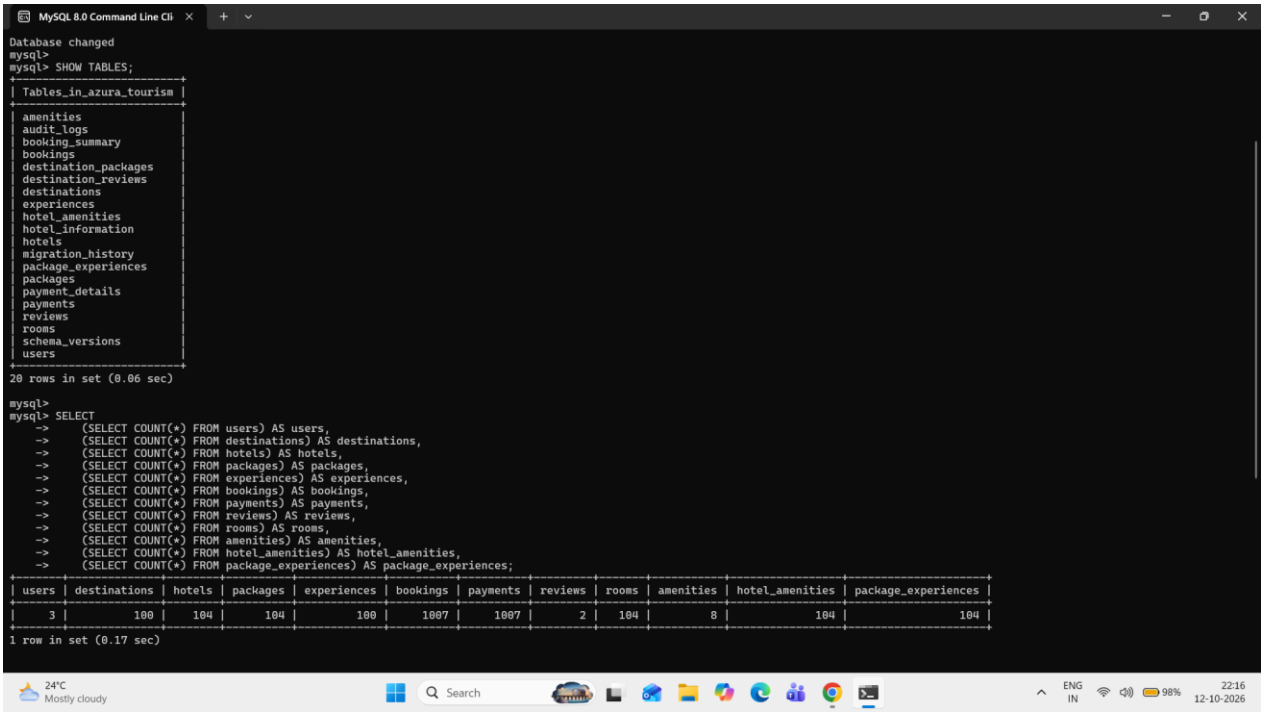


Figure C1. Database object listing and recorded row counts. The visible output shows 100 destinations, 1,007 bookings, 1,007 payments and counts for the other displayed tables.

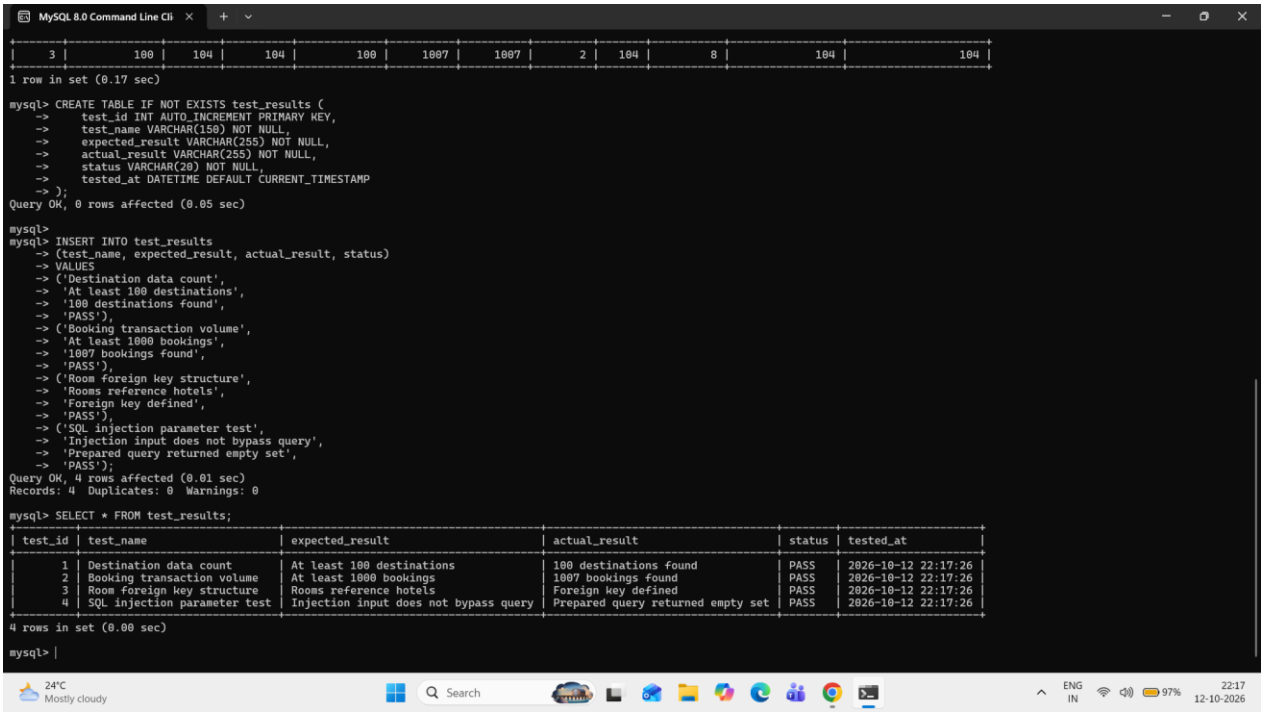


Figure C2. Manual test-results ledger. Four records are marked PASS: destination data count, booking transaction volume, room foreign-key structure and the prepared-query parameter test.

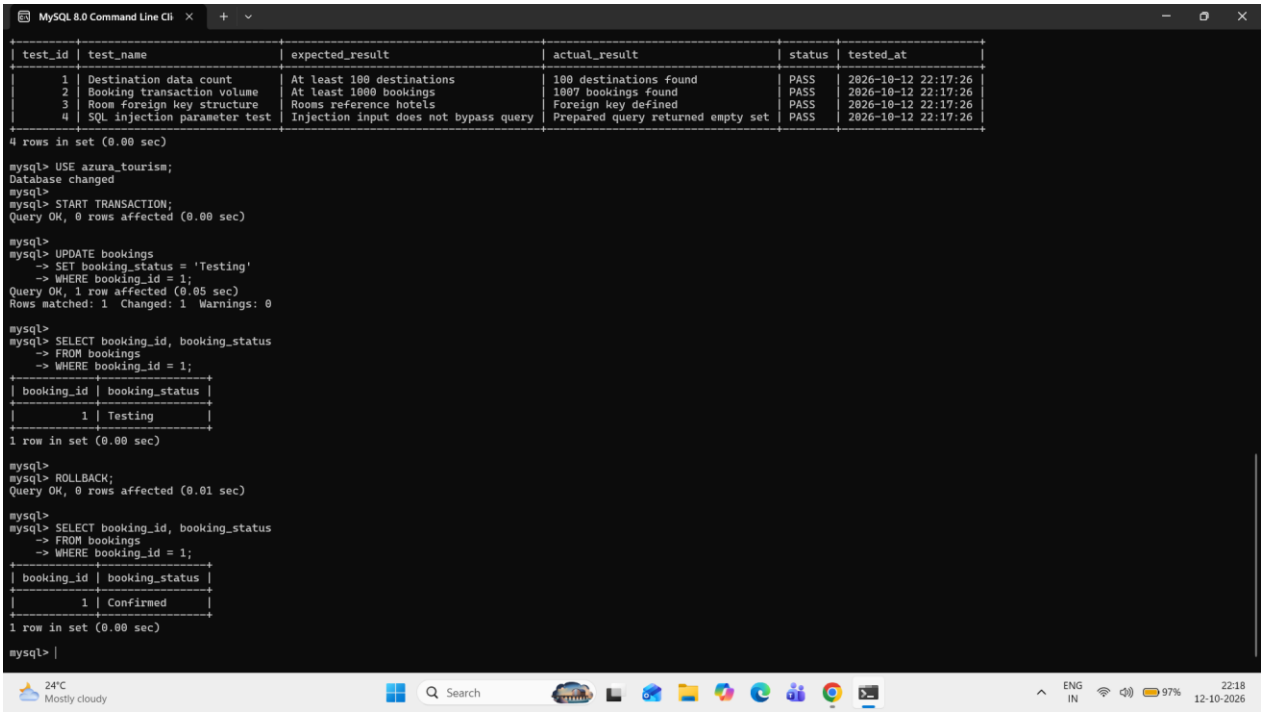


Figure C3. Transaction rollback. Booking 1 is temporarily changed to Testing; after ROLLBACK, the displayed status returns to Confirmed.

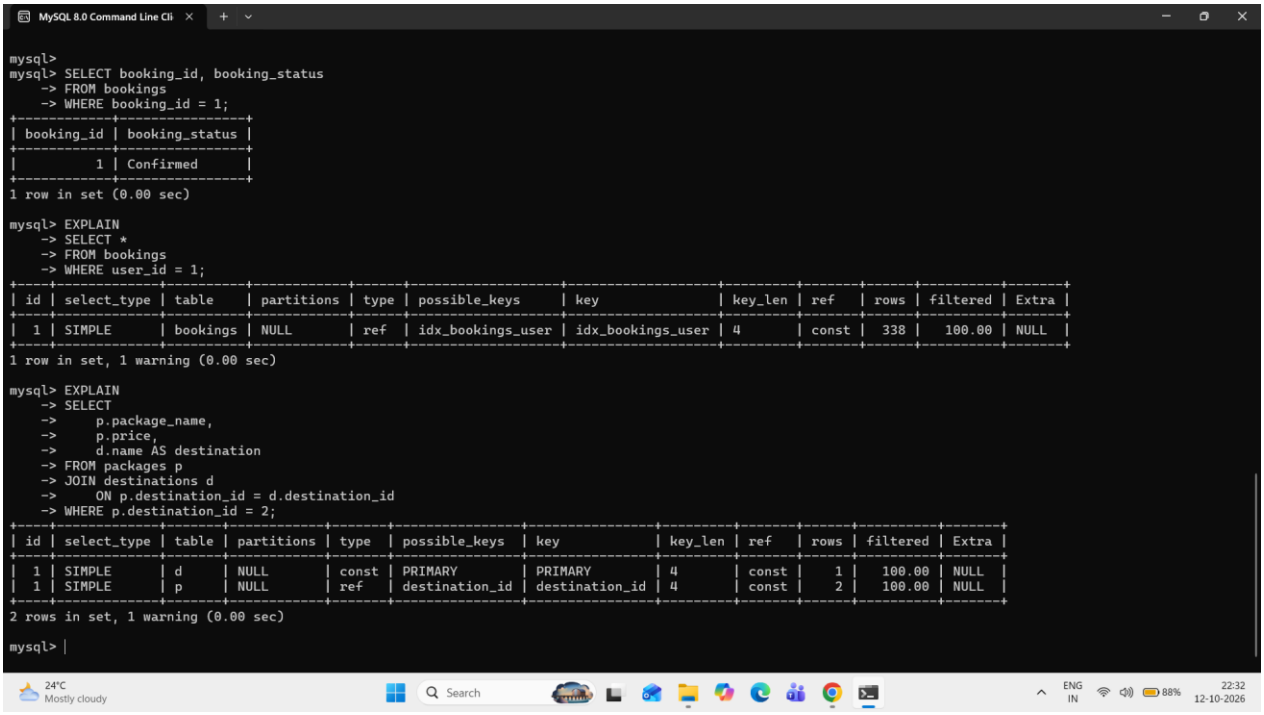


Figure C4. EXPLAIN output. The bookings-by-user query selects idx_bookings_user; the package/destination query shows the primary-key and destination_id indexes. Execution-time before/after comparisons are not shown.

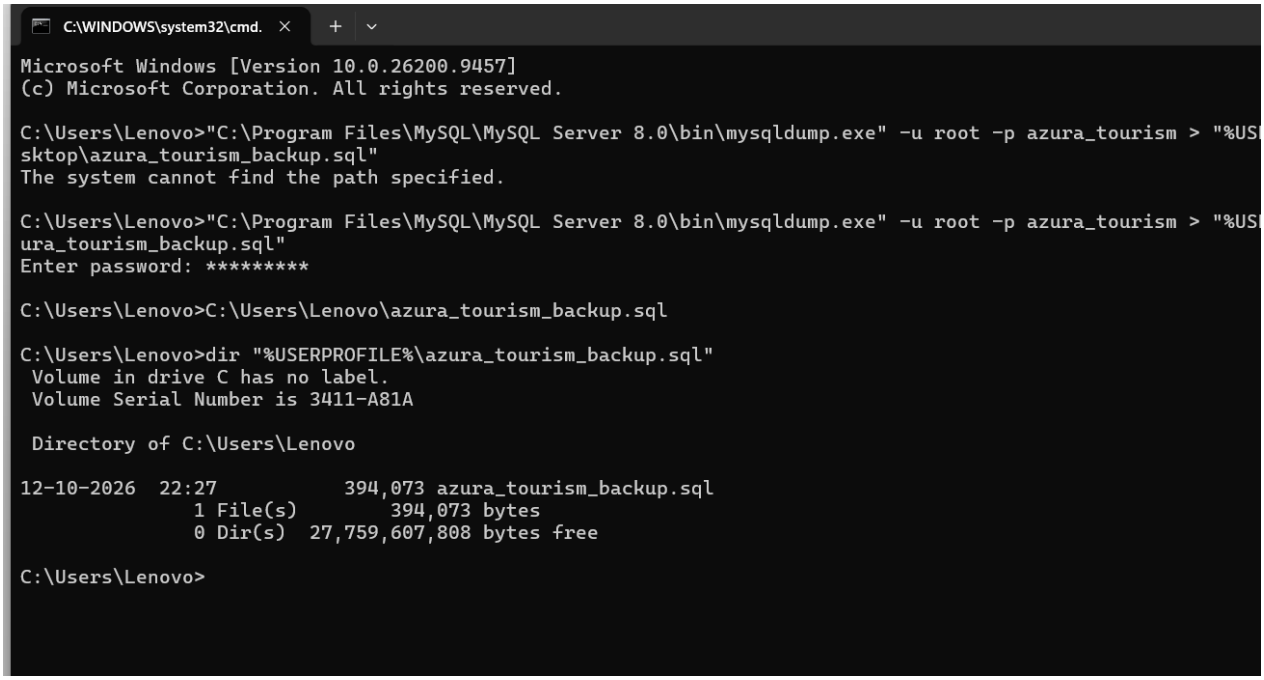


Figure C5. Backup file evidence. The Windows command prompt shows azura_tourism_backup.sql and a displayed file size of 394,073 bytes.

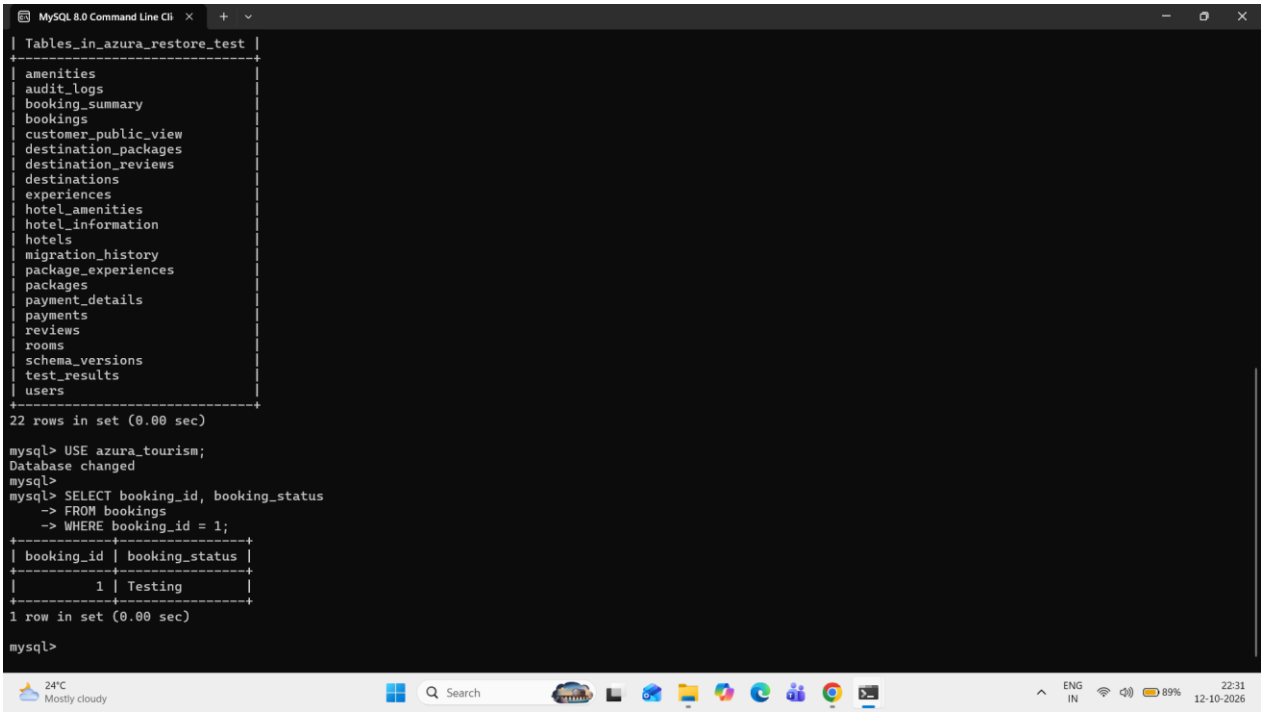


Figure C6. Restored-schema object listing and a booking-status query. This supports that objects were visible in the restore/test workflow; it is not a complete row-by-row backup integrity comparison.

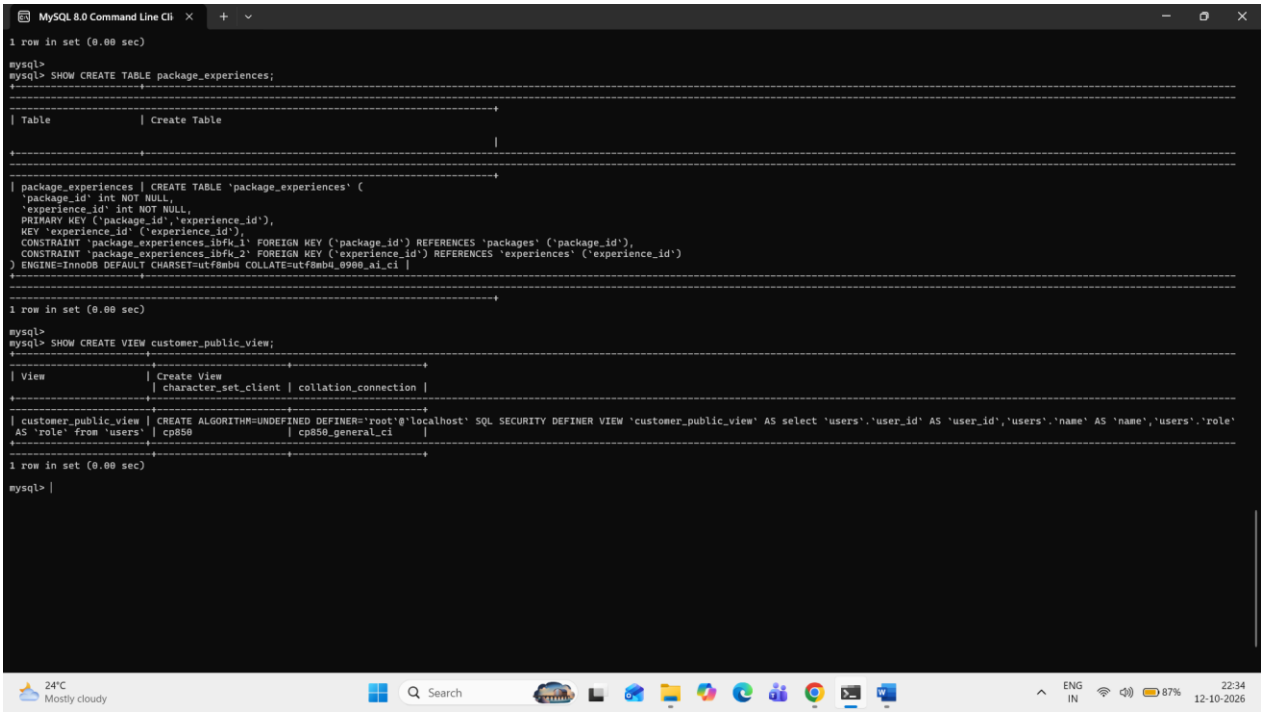


Figure C7. Supporting table definitions and relationship structures, including the package_experiences bridge table. This is implementation evidence and does not replace an updated conceptual ER diagram.

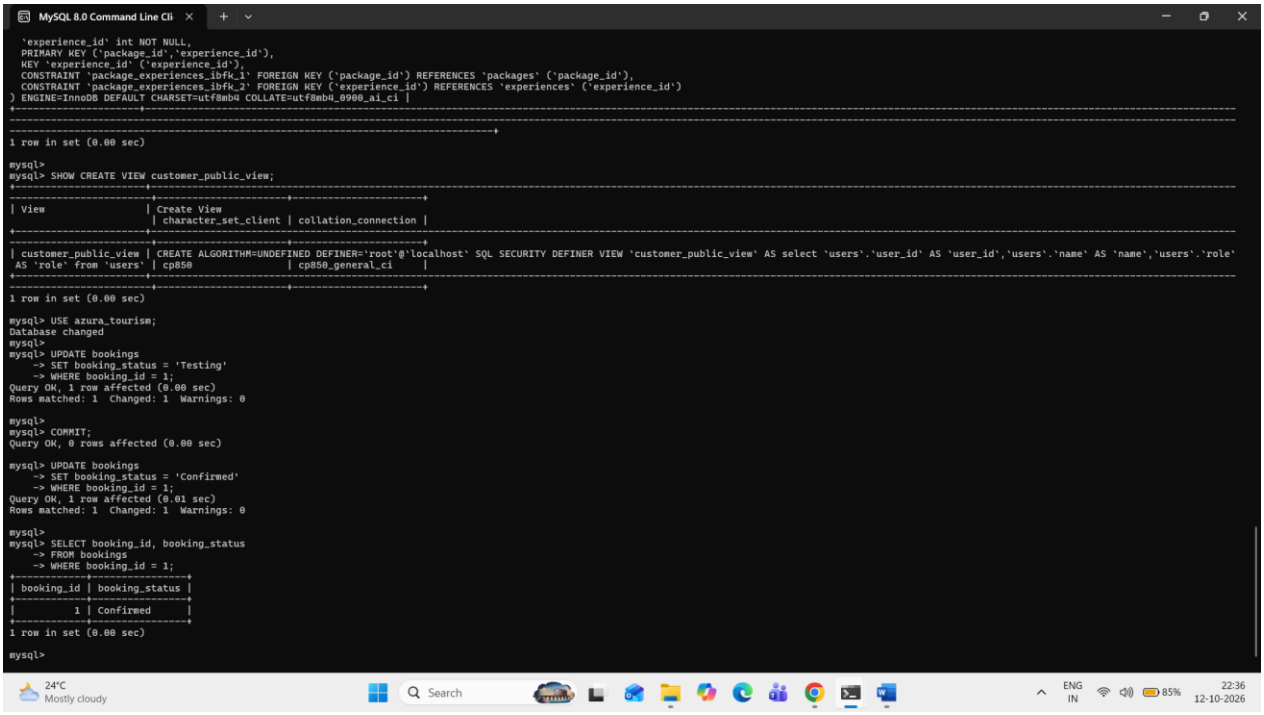


Figure C8. Prepared-statement / restricted-access testing evidence as displayed in the MySQL client. Interpret the screenshot only for the statements and results visible; application-wide security is not proven by one test.

Appendix C.1 — Evidence Gaps

- The supplied screenshots do not demonstrate all 20 required SQL queries.
- The EXPLAIN screenshots do not show five measured before-and-after execution times or percentage improvements.
- The screenshots do not demonstrate the full concurrency/lost-update/deadlock requirement.
- The row-count screenshot shows the displayed tables but does not establish approximately 10,000 total rows.
- The screenshots do not show a complete Git repository with executable migrations, rollback scripts and automated test runner.
- The approved core conceptual ER diagram has eight entities; the physical bridge/dependent tables visible in SQL have not all been reconciled into that diagram.